第九课预习报告

binary.c

- 作用: 演示如何将整数转换为二进制表示。
- 功能: 提示用户输入一个整数并以递归方式打印它的二进制形式。
- 演示效果:

```c
/* binary.c -- prints integer in binary form */
#include <stdio.h>
void to_binary(unsigned long n);

int main(void)
{
    unsigned long number;
    printf("Enter an integer (q to quit):\n");
    while (scanf("%lu", &number) == 1)
    {
        printf("Binary equivalent: ");
        to_binary(number);
        putchar('\n');
        printf("Enter an integer (q to quit):\n");
    }
    printf("Done.\n");

    return 0;
}

void to_binary(unsigned long n)   /* recursive function */
{
    int r;

    r = n % 2;
    if (n >= 2)
        to_binary(n / 2);
     putchar(r == 0 ? '0' : '1');

    return;
}
```

```
gcc binary.c -o binary && echo "20`nq" | ./binary
Enter an integer (q to quit):
Binary equivalent: 10100
Enter an integer (q to quit):
Done.
```

factor.c

- 作用: 展示循环与递归求阶乘的实现方式。
- 功能: 用户输入 0–12 之间的整数，程序分别用循环与递归计算阶乘。
- 演示效果:

```c
// factor.c -- uses loops and recursion to calculate factorials
#include <stdio.h>
long fact(int n);
long rfact(int n);
int main(void)
{
    int num;

    printf("This program calculates factorials.\n");
    printf("Enter a value in the range 0-12 (q to quit):\n");
    while (scanf("%d", &num) == 1)
    {
        if (num < 0)
            printf("No negative numbers, please.\n");
        else if (num > 12)
            printf("Keep input under 13.\n");
        else
        {
            printf("loop: %d factorial = %ld\n",
                   num, fact(num));
            printf("recursion: %d factorial = %ld\n",
                   num, rfact(num));
        }
        printf("Enter a value in the range 0-12 (q to quit):\n");
    }
    printf("Bye.\n");

    return 0;
}
```

```c

long fact(int n)      // loop-based function
{
    long ans;

    for (ans = 1; n > 1; n--)
        ans *= n;

    return ans;
}

long rfact(int n)     // recursive version
{
    long ans;

    if (n > 0)
        ans= n * rfact(n-1);
    else
        ans = 1;

    return ans;
}
```

```
gcc factor.c -o factor && echo "5`nq" | ./factor
This program calculates factorials.
Enter a value in the range 0-12 (q to quit):
loop: 5 factorial = 120
recursion: 5 factorial = 120
Enter a value in the range 0-12 (q to quit):
Bye.
```

swap2.c

- 作用: 进一步验证参数传值调用的行为。
- 功能: 通过打印 `u` 和 `v` 的值变化过程，展示函数内部交换成功但不会影响主函数变量的事实。
- 演示效果:

```c
/* swap2.c -- researching swap1.c */
#include <stdio.h>
void interchange(int u, int v);

```

```c
int main(void)
{
    int x = 5, y = 10;

    printf("Originally x = %d and y = %d.\n", x , y);
    interchange(x, y);
    printf("Now x = %d and y = %d.\n", x, y);

    return 0;
}

void interchange(int u, int v)
{
    int temp;

    printf("Originally u = %d and v = %d.\n", u , v);
    temp = u;
    u = v;
    v = temp;
    printf("Now u = %d and v = %d.\n", u, v);
}
```

```
gcc swap2.c -o swap2 && ./swap2
Originally x = 5 and y = 10.
Originally u = 5 and v = 10.
Now u = 10 and v = 5.
Now x = 5 and y = 10.
```

swap1.c

- 作用: 展示参数传值调用的特性。
- 功能: 尝试在函数中交换两个整数的值，但由于使用了 **值传递**，实际的变量值不会被改变。
- 语义错误:
  1. 函数 `interchange(int u, int v)` 只是交换了副本 `u` 和 `v`，而不是主函数中的 `x` 和 `y`。
  2. 结果显示变量未被成功交换。
- 演示效果:

```c
/* swap1.c -- first attempt at a swapping function */
```

```c
#include <stdio.h>
void interchange(int u, int v); /* declare function */

int main(void)
{
    int x = 5, y = 10;

    printf("Originally x = %d and y = %d.\n", x , y);
    interchange(x, y);
    printf("Now x = %d and y = %d.\n", x, y);

    return 0;
}

void interchange(int u, int v)  /* define function  */
{
    int temp;

    temp = u;
    u = v;
    v = temp;
}
```

```
gcc swap1.c -o swap1 && ./swap1
Originally x = 5 and y = 10.
Now x = 5 and y = 10.
```

recur.c

- 作用: 演示函数递归调用时栈帧的变化。
- 功能: 通过打印变量 `n` 的地址，展示每次递归调用会创建新的栈帧，地址逐层变化，返回时反向输出。
- 演示效果:

```c
/* recur.c -- recursion illustration */
#include <stdio.h>
void up_and_down(int);

int main(void)
{
    up_and_down(1);
```

```c
    return 0;
}

void up_and_down(int n)
{
    printf("Level %d: n location %p\n", n, &n); // 1
    if (n < 4)
        up_and_down(n+1);
    printf("LEVEL %d: n location %p\n", n, &n); // 2

}
```

```
gcc recur.c -o recur && ./recur
Level 1: n location 0000006bd1bffd30
Level 2: n location 0000006bd1bffd00
Level 3: n location 0000006bd1bffcd0
Level 4: n location 0000006bd1bffca0
LEVEL 4: n location 0000006bd1bffca0
LEVEL 3: n location 0000006bd1bffcd0
LEVEL 2: n location 0000006bd1bffd00
LEVEL 1: n location 0000006bd1bffd30
```

proto.c

- 作用: 演示函数原型的使用及参数检查。
- 功能: 定义了带参数的函数原型 `int imax(int, int);`，展示编译器如何检查参数数量和类型的正确性。
- 主要错误:
  1. 调用 `imax(3)` 参数不足，编译器报错“too few arguments”。
  2. 调用 `imax(3.0, 5.0)` 由于参数类型不匹配，虽然浮点数可隐式转换为整数，但不是正确用法。
- 演示效果:

```c
/* proto.c -- uses a function prototype */
#include <stdio.h>
int imax(int, int);        /* prototype */
int main(void)
{
    printf("The maximum of %d and %d is %d.\n",
```

```
            3, 5, imax(3));
    printf("The maximum of %d and %d is %d.\n",
            3, 5, imax(3.0, 5.0));
    return 0;
}

int imax(int n, int m)
{
    return (n > m ? n : m);
}
```

```
gcc proto.c -o proto && ./proto
proto.c: In function 'main':
proto.c:7:18: error: too few arguments to function 'imax'; expected 2, have 1
    7 |            3, 5, imax(3));
      |^~~~
proto.c:3:5: note: declared here
    3 | int imax(int, int);        /* prototype */
      |     ^~~~
```

misuse.c

- 作用: 演示错误使用函数的情况。
- 功能: 定义了一个旧式声明的函数 `imax`，但在调用时传入了错误数量和类型的参数，导致编译错误。
- 主要错误:
  1. 函数声明 `int imax();` 未指定参数类型，编译器认为无参数。
  2. 调用 `imax(3)` 和 `imax(3.0, 5.0)` 与声明不匹配，导致“参数数量不匹配”错误。
- 演示效果:

```
/* misuse.c -- uses a function incorrectly */
#include <stdio.h>
int imax();      /* old-style declaration */

int main(void)
{
    printf("The maximum of %d and %d is %d.\n",
            3, 5, imax(3));
    printf("The maximum of %d and %d is %d.\n",
```

```
            3, 5, imax(3.0, 5.0));
    return 0;
}

int imax(n, m)
int n, m;
{
    return (n > m ? n : m);
}
```

```
gcc misuse.c -o misuse && ./misuse
misuse.c: In function 'main':
misuse.c:8:18: error: too many arguments to function 'imax'; expected 0, have 1
    8 |            3, 5, imax(3));
      |^~~~ ~
misuse.c:3:5: note: declared here
    3 | int imax();      /* old-style declaration */
      |     ^~~~
misuse.c:10:18: error: too many arguments to function 'imax'; expected 0, have 2
   10 |            3, 5, imax(3.0, 5.0));
      |^~~~ ~~~
misuse.c:3:5: note: declared here
    3 | int imax();      /* old-style declaration */
      |     ^~~~
misuse.c: In function 'imax':
misuse.c:14:5: warning: old-style function definition [-Wold-style-definition]
   14 | int imax(n, m)
      |     ^~~~
misuse.c:16:1: error: number of arguments doesn't match prototype
   16 | {
      | ^
misuse.c:3:5: error: prototype declaration
    3 | int imax();      /* old-style declaration */
      |     ^~~~
```

loccheck.c

- 作用: 检查变量在不同函数中的存储位置。
- 功能: 打印变量的值和地址，展示局部变量在不同作用域下的内存位置。

- 演示效果:

```c
/* loccheck.c  -- checks to see where variables are stored  */
#include <stdio.h>
void mikado(int);                       /* declare function  */
int main(void)
{
    int pooh = 2, bah = 5;              /* local to main()   */

    printf("In main(), pooh = %d and &pooh = %p\n",
           pooh, &pooh);
    printf("In main(), bah = %d and &bah = %p\n",
           bah, &bah);
    mikado(pooh);

    return 0;
}

void mikado(int bah)                    /* define function   */
{
    int pooh = 10;                      /* local to mikado() */

    printf("In mikado(), pooh = %d and &pooh = %p\n",
           pooh, &pooh);
    printf("In mikado(), bah = %d and &bah = %p\n",
           bah, &bah);
}
```

```
gcc loccheck.c -o loccheck && ./loccheck
In main(), pooh = 2 and &pooh = 00000013a73ffd9c
In main(), bah = 5 and &bah = 00000013a73ffd98
In mikado(), pooh = 10 and &pooh = 00000013a73ffd5c
In mikado(), bah = 2 and &bah = 00000013a73ffd70
```

lethead2.c

- 作用: 打印居中对齐的公司信头信息。
- 功能: 使用自定义函数打印指定数量的字符，实现居中排版公司名称、地址和所在地。
- 演示效果:

```c
/* lethead2.c */
#include <stdio.h>
#include <string.h>            /* for strlen() */
#define NAME "GIGATHINK, INC."
#define ADDRESS "101 Megabuck Plaza"
#define PLACE "Megapolis, CA 94904"
#define WIDTH 40
#define SPACE ' '

void show_n_char(char ch, int num);

int main(void)
{
    int spaces;

    show_n_char('*', WIDTH);   /* using constants as arguments */
    putchar('\n');
    show_n_char(SPACE, 12);    /* using constants as arguments */
    printf("%s\n", NAME);
    spaces = (WIDTH - strlen(ADDRESS)) / 2;
    /* Let the program calculate    */
    /* how many spaces to skip      */
    show_n_char(SPACE, spaces);/* use a variable as argument   */
    printf("%s\n", ADDRESS);
    show_n_char(SPACE, (WIDTH - strlen(PLACE)) / 2);
    /* an expression as argument    */
    printf("%s\n", PLACE);
    show_n_char('*', WIDTH);
    putchar('\n');

    return 0;
}

/* show_n_char() definition */
void show_n_char(char ch, int num)
{
    int count;

    for (count = 1; count ⩽ num; count++)
        putchar(ch);
}
```

```
gcc lethead2.c -o lethead2 && ./lethead2
****************************************
            GIGATHINK, INC.
           101 Megabuck Plaza
          Megapolis, CA 94904
****************************************
```

lethead1.c

- 作用: 打印公司信头信息。
- 功能: 使用自定义函数打印由星号组成的分隔线，并输出公司名称、地址和所在地。
- 演示效果:

```c
/* lethead1.c */
#include <stdio.h>
#define NAME "GIGATHINK, INC."
#define ADDRESS "101 Megabuck Plaza"
#define PLACE "Megapolis, CA 94904"
#define WIDTH 40

void starbar(void);  /* prototype the function */

int main(void)
{
    starbar();
    printf("%s\n", NAME);
    printf("%s\n", ADDRESS);
    printf("%s\n", PLACE);
    starbar();       /* use the function       */

    return 0;
}

void starbar(void)   /* define the function    */
{
    int count;

    for (count = 1; count <= WIDTH; count++)
        putchar('*');
    putchar('\n');
```

```
}
```

```
gcc lethead1.c -o lethead1 && ./lethead1
****************************************
GIGATHINK, INC.
101 Megabuck Plaza
Megapolis, CA 94904
****************************************
```

hotel.c + usehotel.c

- 作用: 模拟酒店房价计算系统。
- 功能: 用户选择酒店并输入入住天数，程序根据折扣规则计算总价。
- 演示效果:

```c
/* hotel.c -- hotel management functions */
#include <stdio.h>
#include "hotel.h"
int menu(void)
{
    int code, status;

    printf("\n%s%s\n", STARS, STARS);
    printf("Enter the number of the desired hotel:\n");
    printf("1) Fairfield Arms           2) Hotel Olympic\n");
    printf("3) Chertworthy Plaza        4) The Stockton\n");
    printf("5) quit\n");
    printf("%s%s\n", STARS, STARS);
    while ((status = scanf("%d", &code)) != 1  ||
           (code < 1 || code > 5))
    {
        if (status != 1)
            scanf("%*s");   // dispose of non-integer input
        printf("Enter an integer from 1 to 5, please.\n");
    }

    return code;
}

int getnights(void)
{
```

```
    int nights;

    printf("How many nights are needed? ");
    while (scanf("%d", &nights) != 1)
    {
        scanf("%*s");       // dispose of non-integer input
        printf("Please enter an integer, such as 2.\n");
    }

    return nights;
}

void showprice(double rate, int nights)
{
    int n;
    double total = 0.0;
    double factor = 1.0;

    for (n = 1; n <= nights; n++, factor *= DISCOUNT)
        total += rate * factor;
    printf("The total cost will be $%0.2f.\n", total);
}

/* usehotel.c -- room rate program */
#include <stdio.h>
#include "hotel.h" /* defines constants, declares functions */

int main(void)
{
    int nights;
    double hotel_rate;
    int code;

    while ((code = menu()) != QUIT)
    {
        switch(code)
        {
            case 1 : hotel_rate = HOTEL1;
                break;
            case 2 : hotel_rate = HOTEL2;
                break;
            case 3 : hotel_rate = HOTEL3;
                break;
            case 4 : hotel_rate = HOTEL4;
```

```
                break;
            default: hotel_rate = 0.0;
                printf("Oops!\n");
                break;
        }
        nights = getnights();
        showprice(hotel_rate, nights);
    }
    printf("Thank you and goodbye.\n");

    return 0;
}
```

```
gcc usehotel.c hotel.c -o usehotel && echo "2`n3`n5" | ./usehotel
**********************************************************************
Enter the number of the desired hotel:
1) Fairfield Arms           2) Hotel Olympic
3) Chertworthy Plaza        4) The Stockton
5) quit
**********************************************************************
How many nights are needed? The total cost will be $641.81.

**********************************************************************
Enter the number of the desired hotel:
1) Fairfield Arms           2) Hotel Olympic
3) Chertworthy Plaza        4) The Stockton
5) quit
**********************************************************************
Thank you and goodbye.
```

lesser.c

- 作用: 比较两个整数并返回较小值。
- 功能: 用户输入两个整数，程序输出其中较小的一个。
- 演示效果:

```
/* lesser.c -- finds the lesser of two evils */
#include <stdio.h>
int imin(int, int);

int main(void)
```

```c
{
    int evil1, evil2;

    printf("Enter a pair of integers (q to quit):\n");
    while (scanf("%d %d", &evil1, &evil2) == 2)
    {
        printf("The lesser of %d and %d is %d.\n",
               evil1, evil2, imin(evil1,evil2));
        printf("Enter a pair of integers (q to quit):\n");
    }
    printf("Bye.\n");

    return 0;
}

int imin(int n,int m)
{
    int min;

    if (n < m)
        min = n;
    else
        min = m;

    return min;
}
```

```
gcc lesser.c -o lesser && echo "8 3`nq" | ./lesser
Enter a pair of integers (q to quit):
The lesser of 8 and 3 is 3.
Enter a pair of integers (q to quit):
Bye.
```